



P.O.O

Programación orientado a objetos

ENCAPSULAMIENTO

- Consiste en almacenar y organizar en una clase, las características y funcionalidades de los objetos, representando las por medio de atributos y métodos.



Gracias al encapsulamientos podemos asegurar que los datos sean correctos y completos y además evita el acceso de datos por otro medio que sea distinto al que indiquemos.

- Gracias a eso podemos controlarlo gracias a los modificadores de acceso:
 - **Público:** todos tienen permiso para modificar.
 - **Protected:** acceso a elementos de la misma clase o paquetes.
 - **Private:** sólo puede ser accedido por los métodos o constructores de la misma clase.

En resumen, los modificadores de acceso, nos permiten generar un nivel de seguridad mayor en nuestras clases.

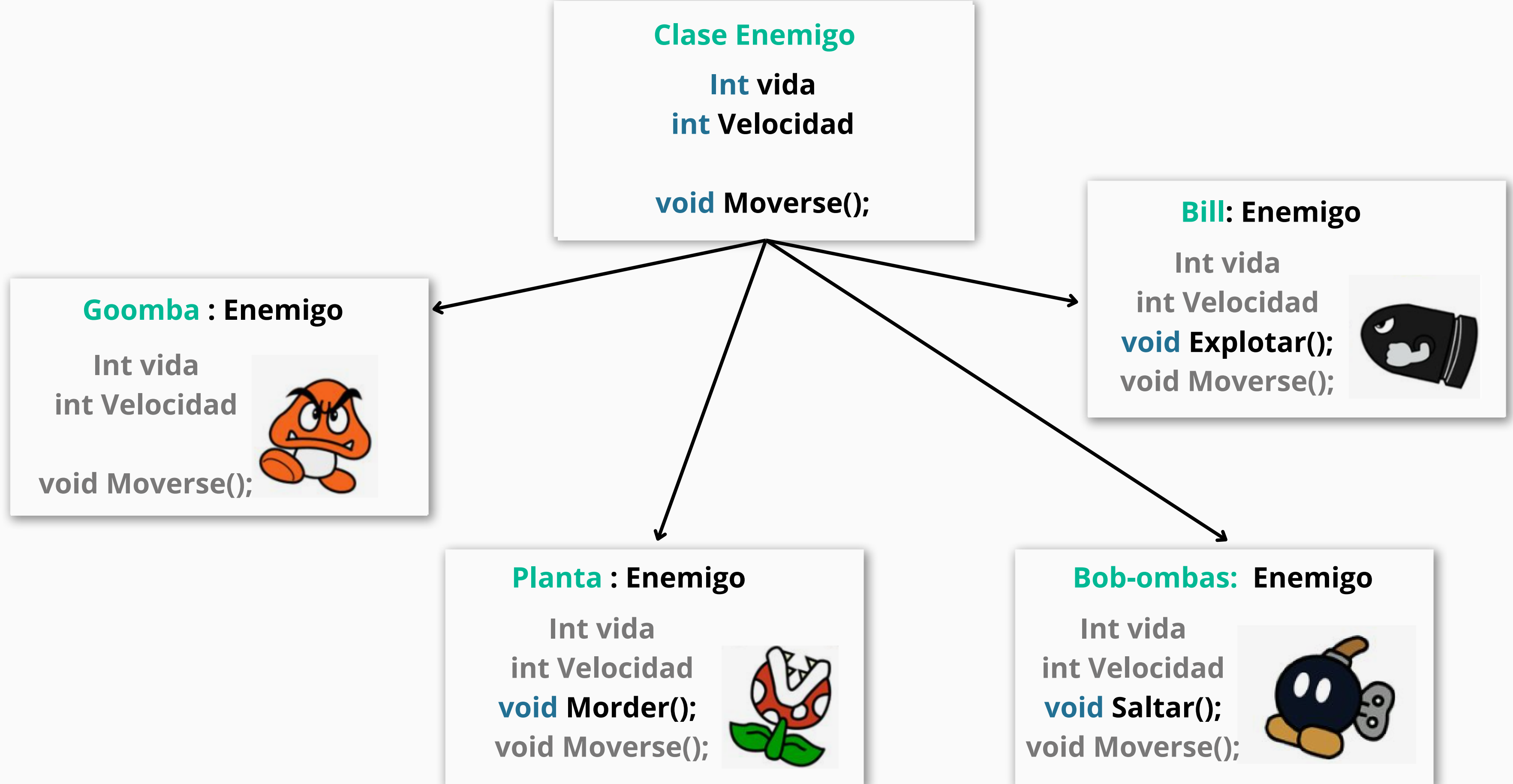
HERENCIA

- La herencia permite establecer una estructura de clases organizada en jerarquías.
- En las clases superiores (conocidas como super clases) se definen las propiedades y el comportamiento más general y se crean clases hijas (subclases) que heredan las propiedades y el comportamiento de la superclase.
- A través de la herencia se pueden construir clases cada vez más específicas pero que tienen características en común entre ellas ya que descienden de la misma superclase

HERENCIA

Clase Enemigo





ABSTRACCIÓN

- Es un proceso por lo cual se descarta aquella información que no resulta relevante en un contexto actual, depende principalmente el criterio del observador (programador), la abstracción y la capacidad de captar las distintos atributos y métodos que se van a utilizar en el juego.
- Al definir una clase podemos declarar que esta es abstracta. Una clase abstracta no puede ser instanciada, sirve para hacer de tipo base para otras clases. Sin embargo, puede tener constructores declarados con el mismo propósito.

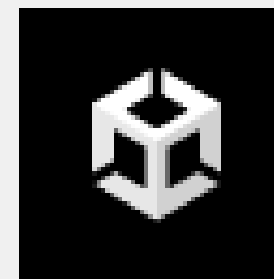
DECLARACIÓN DE UN CLASE ABSTRACTA

Podemos definir que nuestra clase y métodos sean abstractos de la siguiente manera.

```
1  using System;
2  using System.Collections;
3  using System.Collections.Generic;
4  using UnityEngine;
5
6  public abstract class Enemy : MonoBehaviour
7  {
8      public int Vida;
9      public float Velocidad;
10
11     0 referencias
12     public abstract void Atacar();
13
14     0 referencias
15     public void Defender()
16     {
17         Debug.Log("El Enemigo se defendió.");
18     }
19 }
```

Toda clase que sea abstracta no puede ser instanciada.

Can't add script



Can't add script behaviour 'Enemy'. The script class can't be abstract!

OK

POLIMORFISMO

- El polimorfismo nos permite tomar un conjunto de objetos de distinto tipo y ejecutar uno o más métodos que todos ellos entienden, pero la función que ejecute ese método será propia de cada tipo de objeto.
- Esto nos permite crear distintos tipos de objetos que respondan de manera distinta a un mismo estímulo.
- Este es uno de los conceptos más poderosos de la POO.

DECLARACIÓN DE UN MÉTODO CON POLIMORFISMO

```
Script de Unity | 3 referencias
public abstract class PowerUp : MonoBehaviour
{
    protected string nombre;
    public Mario player;

    Mensaje de Unity | 0 referencias
    private void Start()
    {
        if(player != null)
        {
            player = GetComponent<Mario>();
        } else
        {
            player = FindObjectOfType<Mario>();
        }
    }

    3 referencias
    public abstract void Consumible();
}
```

Si queremos aplicar polimorfismo a un método en específico dentro de nuestra clase lo podemos declarar de la siguiente dos formas:

1. **public virtual void** "NombreDelMetodo"().. : esta forma lo utilizamos cuando tenemos una lógica previa que queramos re utilizar.

```
public virtual void ActivarPowerUp() {
    debug.log("Activar Power UP");
}
```

1. **public abstract void** "NombreDelMetodo"(); si no tenemos una lógica definida previamente.

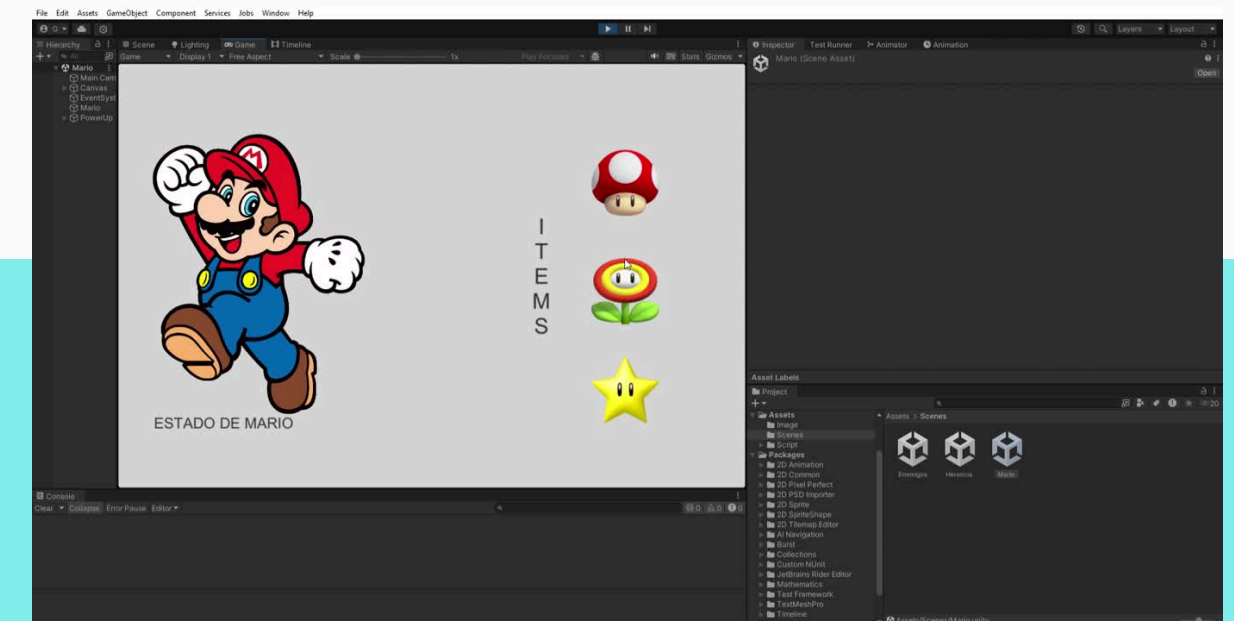
DECLARACIÓN DE UN MÉTODO CON POLIMORFISMO

```
Script de Unity (1 referencia de recurso) | 0 referencias
public class Estrella : PowerUp
{
    2 referencias
    public override void ActivarPowerUp()
    {
        base.ActivarPowerUp();
    }
    1 referencia
    public override void Consumible()
    {
        player.Cambio_de_Estado(Mario.MarioState.estrella);
    }
}
```

Para realizar el polimorfismo, es necesario heredar la clase previamente.

Se utiliza **public override void** “NombreDelmetodo” para sobrescribir el método que estamos heredando, para así, poder modificar la lógica a nuestro gusto.

Se utiliza **base.NombreDelMetodo();** para re utilizar la lógica utilizada en nuestra clase padre.





CONCLUSIÓN

